CHAPTER 12 PERFORMANCE / EVIDENCE / CREDITS

Query Performance and Warehouse Optimization

Make the query correct; diagnose the bottleneck; then spend deliberately

The tempting fix is to double the warehouse.
At 08:30 a Power BI report that normally opens promptly starts taking twenty seconds. The data engineer increases the Snowflake warehouse size. The next screen loads quickly, but the team cannot say whether resizing helped: the first request resumed a suspended warehouse, the second benefited from cache, and the underlying SQL unexpectedly duplicates generation rows after joining a many-to-many lookup. Credits rise while the KPI remains wrong. A faster answer is not an improved system when it answers the wrong question.

This chapter provides an investigation method for the *Finland Energy Analytics* platform. Building on Chapters 2, 4 and 11, it connects the semantics of a query with evidence from Query History, execution operators, micro-partition pruning, queuing, memory pressure and measured cost. Optimisation is presented as an experiment, not as a collection of magic switches.

Learning outcomes
You will: (1) formulate falsifiable tuning hypotheses; (2) compare the same correct result before and after a change; (3) interpret Query Profile and history fields; (4) decide whether pruning, warehouse sizing, clustering, caches, query acceleration or search optimisation addresses the observed bottleneck; (5) report latency and credits honestly; and (6) explain why a result from a six-row teaching fixture cannot predict production performance.

Prerequisites. Chapters 2, 4 and 11; SQL `JOIN`, `GROUP BY` and time filters; access to a permitted Snowflake test environment for optional cloud runs. Offline lab tests use only synthetic data. Cloud benchmarking may consume credits; agree a budget and use a restricted role.

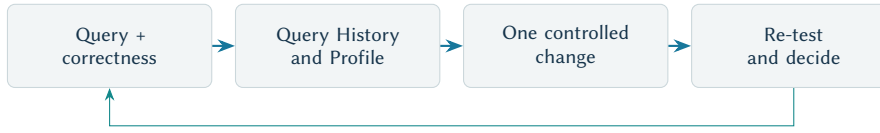


Figure 12.1. Performance work is an evidence loop, not a warehouse-size reflex.

12.1 A structured performance troubleshooting workflow

12.1.1 Hypotheses and reproducible tests

Performance work begins with a statement that can be disproved. "Snowflake is slow" is not testable. "The daily energy report scans most partitions because its timestamp predicate cannot exploit the stored range" is testable: inspect scan ratios, rewrite the filter without changing its meaning, and compare profiles over repeated runs. Record the query text or versioned file, query ID, QUERY_TAG, warehouse size, account/region, inputs, role, result shape, cache policy and competing workload.

A useful triage sequence is **scope** → **correctness** → **wait time** → **operators** → **candidate change** → **repeat**. Distinguish a slow visual from a slow Snowflake statement: report rendering, gateway/network transfer, source SQL compilation, warehouse provisioning and query execution contribute separately to perceived latency. Do not optimise a warehouse on the basis of an uncorrelated stopwatch from the browser.

Listing 1: Illustrative session controls: do not change shared warehouses.

```
ALTER SESSION SET QUERY_TAG =
  'book_ch12_energy_benchmark_v1';
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
SELECT CURRENT_ROLE(), CURRENT_WAREHOUSE();
-- Restore at the end of the isolated experiment:
ALTER SESSION UNSET QUERY_TAG;
ALTER SESSION UNSET USE_CACHED_RESULT;
```

Disabling persisted query-result reuse does *not* eliminate warehouse-local data caching or other environmental variation. A warm-versus-cold experiment must record warehouse suspension/resume state and cannot assert perfect cache isolation without further controls.

Experimental contract
Question: does a half-open timestamp range reduce work compared with an expression around the filtered column? **Invariant:** identical grouping keys and MWh totals. **Evidence:** query IDs, scan/partition counts, execution and queuing components, five or more controlled repeats per treatment. **Decision:** accept only when gains are repeatable and materially exceed operational costs.

Project reference: P017: sfc-gh-bokon/query_performance_deep_dive. *Not independently executed; rights review pending.*

Check. What observation would falsify a theory that pruning is responsible for the delay?

12.1.2 Correctness before speed

An optimisation may reduce runtime while changing the answer. Consider the running case: six accepted technology observations sum to **66.000 MWh** for one synthetic UTC interval. If an engineer joins those rows to a technology alias table with two matching aliases per row, a subsequent `SUM` can silently double production to **132.000 MWh**. Neither a faster join algorithm nor more compute makes that result acceptable.

Listing 2: Confirm the gold grain before joining.

```
SELECT interval_utc, technology, COUNT(*) AS n
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
GROUP BY interval_utc, technology
HAVING COUNT(*) <> 1;
-- Expected on the provided seven-day fixture: 0 rows.
```

For each baseline and proposed rewrite, test **row count, key uniqueness, null handling, unit consistency and aggregates**. A missing period must not become a zero-valued period unless the business contract explicitly defines imputation. When the source includes revisions, benchmark the deduplicated authoritative state, not all raw deliveries. Compare all result rows (or a deterministic keyed digest) rather than only a total that could conceal offsetting errors.

Two kinds of correctness.
Semantic correctness means that generation is measured in MWh, at the intended event-time grain, for the intended geographic and source scope. *Mechanical correctness* means that the query respects uniqueness, grouping and equality of output for a given snapshot. Both must hold.

A practical review pairs each expected result with its computation: `COUNT(*)=42` and `SUM(energy_mwh)=462.000` for the companion's *seven-day expanded synthetic fixture* (six technologies repeated across seven days). These numbers are offline oracle values, not observed Finland generation. **Failure diagnosis.** If a rewritten query disagrees, stop timing tests and inspect join multiplicity, time boundaries, `NULL` behaviour and source versions. **Check.** Why is matching total MWh alone not enough to prove two queries equivalent?

12.1.3 Isolating the performance bottleneck

A single elapsed-time number conceals different problems. A statement can wait for a suspended warehouse to resume (provisioning), queue because concurrency exceeds available capacity (overload), compile complex SQL, scan unnecessary partitions, spill intermediate data or transfer a large result across the network. These demand different remedies. Use the *highest-cost observed stage*, not a guessed bottleneck, to choose the next experiment.

Evidence	Plausible cause	First test
Provisioning wait	Suspended/cold warehouse	Separate resume from steady-state
Overload wait	Concurrent queries	Repeat with controlled concurrency
Many partitions scanned	Weak filter pruning	Preserve sargable ranges
Remote spill	Large intermediate state	Inspect join/aggregation before resizing
Large returned bytes	Excess columns/rows	Return only required fields

Not every indicator is a proof. For example, some workloads legitimately scan nearly all partitions; selective pruning is not meaningful for an unfiltered whole-table aggregation. Similarly, remote spill is usually suspicious but particular services may show nonzero remote-storage metrics for reasons that require profile-level verification. Query compilation can vary independently of scan efficiency.

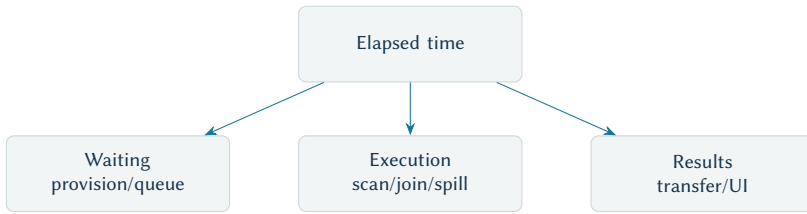


Figure 12.2. One elapsed time may hide three different interventions.

Decision rule. Rewrite an inefficient expression before buying a larger warehouse. Investigate a concurrency bottleneck before changing clustering. Analyse an oversized BI result before enabling a paid acceleration feature. **Check.** Which metric distinguishes queued overload from a long-running hash aggregation?

12.2 Query History and Query Profile

12.2.1 Query History metadata

Snowflake provides recent query details in Snowsight and Information Schema table functions, while SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY supports broader historical analysis subject to privileges and reporting latency. Useful fields include query ID and tag, warehouse name/size, status, total elapsed/compilation/execution times, provisioning and overload waits, scanned bytes, scanned/total partitions, result-cache percentages and spill metrics. Field availability and interpretation should be rechecked at publication time.

Listing 3: Recent per-user history: permitted users only.

```
SELECT query_id, query_tag, warehouse_name,
       total_elapsed_time, execution_time,
       queued_overload_time, bytes_scanned
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(
  DATEADD('HOUR', -1, CURRENT_TIMESTAMP()),
  CURRENT_TIMESTAMP(), 100))
WHERE query_tag = 'book_ch12_energy_benchmark_v1'
ORDER BY start_time DESC;
```

Account Usage can lag behind the most recent execution. For an immediate diagnosis, inspect Snowsight and the appropriate recent-history function; for a retrospective account-wide trend, use a granted monitoring role. Access to observability data is *not* a reason to grant ACCOUNTADMIN to a lab user.

12.2.2 Query Profile operators and scans

Open the saved query ID in Query Profile. Read its execution graph from scans through joins and aggregates to the final result. Compare *actual scanned partitions*, not the verbal presence of a filter. An explicit filter may still scan most table storage when values overlap broadly between micro-partitions; a low scan fraction may already indicate good pruning. Observe rows entering and leaving nodes: a join expanding six rows into twelve can be the root cause of an incorrect total.

Project reference: P018: Snowpark Python performance tips. *Not independently executed; rights review pending.*

Check. Why must the time at which Query History becomes visible be recorded separately from the execution time?

12.2.3 Reading spilled data and queuing indicators

Spill occurs when an operation cannot retain all required intermediate state in available memory. Storage spill to local disk is usually less damaging than remote spill. Sorting, large joins, distinct aggregations and skewed groups are common places to investigate. A larger warehouse may supply more memory, but fixing a cross join or reducing the intermediate working set can be both faster and cheaper. Check the operator-level profile and compare row expansion before resizing.

Queuing is not execution. A query can run in a few hundred milliseconds once scheduled yet wait several seconds behind other requests. Record provisioning wait and overload wait separately from execution; a multi-cluster warehouse primarily addresses concurrent workload throughput where supported, not necessarily the runtime of one serial query. Do not conclude that a clustering key cures queues.

Listing 4: Historical diagnostic fields; requires monitoring privileges.

```
SELECT query_id, warehouse_name,
       total_elapsed_time / 1000.0 AS elapsed_s,
       queued_overload_time / 1000.0 AS overload_s,
       bytes_spilled_to_local_storage AS spill_local,
       bytes_spilled_to_remote_storage AS spill_remote,
       partitions_scanned, partitions_total
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_tag = 'book_ch12_energy_benchmark_v1'
ORDER BY start_time DESC LIMIT 25;
```

Reading a profile without false certainty
Observed: heavy remote spill and large rows entering an aggregation. **Hypothesis:** the intermediate result is too large for the available memory. **Next test:** remove an unintended duplicate join and compare spill and output equality. **Alternative:** if the work is intrinsically memory intensive, compare warehouse sizes under the same workload.

The companion includes an *illustrative metrics CSV* solely to exercise reporting logic. Its invented numbers must never be presented as live Query History measurements or as an empirically proven speedup.

Project reference: P135: Query Performance Streamlit in Snowflake Notebooks. *Not independently executed; rights review pending.*

Check. What evidence would make increasing memory a better option than rewriting the join?

12.3 Micro-partition pruning and clustering

12.3.1 Micro-partition elimination

Snowflake automatically organises table data into micro-partitions and records metadata that the optimiser may use to avoid irrelevant partitions. Pruning works well when a predicate reduces the required value range and stored partition ranges support that exclusion. A filter on `interval_utc` is a candidate when the fact is roughly ordered by time; a filter on an unrelated low-selectivity dimension may scan similar data to the unfiltered query.

Listing 5: Same intended UTC interval; inspect plans, not timings alone.

```
-- Expression around column: may impede pruning.
SELECT SUM(energy_mwh)
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE TO_DATE(interval_utc) = '2026-10-04';

-- Explicit half-open range preserves boundary intent.
SELECT SUM(energy_mwh)
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE interval_utc >= '2026-10-04'::TIMESTAMP_NTZ
  AND interval_utc <  '2026-10-05'::TIMESTAMP_NTZ;
```

Both statements intentionally use a UTC-encoded `TIMESTAMP_NTZ` fixture; in a real pipeline verify the source timestamp type and time-zone contract. The rewrite should return the same six records and **66.000 MWh** on the sample for 4 October. Actual Snowflake optimisation is workload-dependent; an apparent improvement in a seven-day toy table is not convincing evidence.

12.3.2 Clustering key trade-offs

Explicit clustering is generally for sufficiently large tables with repetitive selective filters and demonstrably poor natural clustering. Keys can aid pruning, but maintenance consumes compute and storage; high-cardinality timestamps may be more expensive to maintain than a date-level expression. Avoid clustering every small table, and do not apply a key to our tiny teaching fixture.

Decision before DDL.
Only propose `ALTER TABLE ... CLUSTER BY` after baseline profile evidence, an account-level cost estimate, representative query frequency and written approval. New clustering maintenance costs must be counted alongside per-query gains.

Check. Why can a timestamp predicate be selective at the row level yet show weak partition elimination?

12.3.3 Measuring pruning and maintenance overhead

A useful pruning ratio is $P =$ partitions scanned/partitions total for the *relevant scan*, with the caveat that query-wide totals can aggregate several table scans. Compare the same fact scan's numerator and denominator before and after a controlled change. A decrease from 900/1000 to 120/1000 would be evidence of improved pruning; it is *illustrative*, not a reading from this book's account.

Snowflake's system functions for clustering information can help examine table clustering characteristics where applicable. Such information must be interpreted in the context of selective query shapes and table size. Better clustering depth is not synonymous with lower total spend: frequent ingestion or updates may increase background maintenance credits. Evaluate net savings over the expected workload window, not just the fastest cherry-picked SQL statement.

Scenario	Likely action	Evidence needed
Small seven-day fixture	No clustering change	Correctness and pedagogical trace
Large, time-ordered fact	Test range filter first	Low pruning ratio already?
Large, scrambled, repeated scans	Consider cluster key	Measured savings vs. upkeep
Frequent append/corrections	Count maintenance cost	Volume, DML pattern and spend

Listing 6: Read-only partition history fields (approved role).

```
SELECT query_id, partitions_scanned,
       partitions_total, bytes_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_tag = 'book_ch12_energy_benchmark_v1'
  AND partitions_total IS NOT NULL
ORDER BY start_time DESC;
```

Failure case. An analyst adds clustering, reruns exactly one query, and attributes a short runtime to clustering without checking result cache, warehouse start-up or data snapshot. That is a before/after anecdote, not a reliable experiment.

Project reference: P136: clustering and materialised view comparison. *Not independently executed; rights review pending.*

Check. What makes a maintenance-heavy cluster key economically irrational even if it reduces p95 latency?

12.4 Warehouse sizing and concurrency management

12.4.1 Warehouse size versus throughput

Resizing a warehouse changes the compute and memory available for supported workloads, but not every query scales proportionally. Highly parallel scans, large joins and spill-heavy transformations can benefit; compile-bound statements, tiny queries and serial bottlenecks may not. Compare the same source snapshot, query mix and controlled session conditions under distinct warehouse sizes. Record the minimum billing behavior, auto-suspend settings and exact account pricing rather than inserting universal euro amounts.

The *engineering objective* is a service-level target at a defensible cost, not the shortest possible runtime. A twice-as-large warehouse that halves execution time may have similar theoretical execution credits, but real invoices depend on billing windows, idling, concurrency and auxiliary service charges. In addition, warehouse size is not an intrinsic guarantee of query correctness.

12.4.2 Multi-cluster and queueing

When many Power BI interactions arrive together, a warehouse may queue queries even though each individual query is efficient. Multi-cluster configurations, where the feature is available for the customer's edition, can add capacity for parallel demand. This is a concurrency strategy rather than a way to make one query automatically run faster; sizing the base cluster and optimising generated SQL remain relevant.



Figure 12.3. Scaling concurrent admission is different from shortening one scan.

Failure case. A BI dashboard uses seven visuals, each generating SQL. Fifty active users can create a much more demanding query rate than fifty visual pages alone imply. Measure realistic interaction sequences and cache behaviour. **Check.** Why might an extra cluster reduce queue wait without changing the profile of an individual aggregation?

12.4.3 Cost-normalised capacity planning

A good capacity study includes both *latency distribution* and *credits over the observation window*. For a workload with N completed requests, record throughput as N/T and compute spend as C/N credits per successful request. If requests have different complexity, report each family separately; otherwise a changing mixture can falsely suggest a platform regression.

Snowflake's warehouse metering is not identical to per-query cost attribution. Credits billed during idle time, warehouse provisioning, shared concurrency and optional services complicate simple allocation. If you use query-level attributed compute, state exactly which Account Usage view was queried, its data latency and which cost classes are excluded. Do not infer full invoice totals from elapsed seconds alone.

Example (fully hypothetical, not measured)
At the same workload mix, configuration A serves 100 successful requests in ten minutes at 0.20 credits for the measured interval, while B serves 160 at 0.36 credits. A: 0.002 credits/request; B: 0.00225. B offers higher throughput at a higher average credit cost per request. The choice depends on p95 target and capacity need, not a simplistic winner label.

A disciplined load test increases concurrency in stages, randomises query order, keeps a consistent data snapshot and monitors queueing as well as execution. Do not conduct a load test on a shared production warehouse without operational approval. Add auto-suspend and a spending cap in a dedicated experiment plan.

Metric	What it says	What it cannot prove
p95 wall time	User-facing tail	Which phase is slow
Execution time	Snowflake work duration	External report/render time
Queued overload	Capacity contention	That query SQL is inefficient
Credits/request	Normalised spend	Full cost without idle/services

Project reference: P137: warehouse and SQL tuning reference. *Not independently executed; rights review pending.*

Check. What workload characteristic must stay stable when comparing credits per query?

12.5 Caching behavior and materialized results

12.5.1 Result cache and warehouse-local cache

Snowflake can reuse a persisted query result when eligibility conditions are satisfied. The warehouse also has a local data cache that may accelerate reads after earlier scans. These are different mechanisms with different conditions, and neither is equivalent to a precomputed business mart. A query returning an already cached result can appear exceptionally fast while revealing almost nothing about the original scan's cost.

12.5.2 Controlled warm/cold experiments

For a baseline scan study, explicitly record whether USE_CACHED_RESULT is enabled and repeat measurements with the same policy. Even when persisted-result reuse is disabled, warehouse-local cache, data state and concurrency may vary. A "cold run" after resume and a "warm run" should be labelled operational scenarios, not interchangeable treatments. Alternate baseline and candidate query order to limit drift and do not throw away an inconvenient slow run without a documented reason.

Listing 7: An isolated, reversible experimental session.

```
ALTER SESSION SET USE_CACHED_RESULT=FALSE;
SELECT technology, SUM(energy_mwh)
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
GROUP BY technology;
ALTER SESSION UNSET USE_CACHED_RESULT;
```

12.5.3 Materialization versus recomputation

A materialised result is a design choice: precompute when the same expensive transformation is repeatedly requested, and when refresh semantics and maintenance spending are justified. Options include a scheduled Gold table, a materialized view where SQL/function restrictions allow, or a Dynamic Table with supported refresh behavior. Unlike a cached query result, these objects may incur ongoing refresh costs and operational ownership obligations.

Do not cache a semantic error.
Preaggregating twelve raw revisions into a fast BI mart can make a wrong number arrive sooner. Materialization must occur after the revision-resolution and grain checks established in earlier chapters.

The six-row one-interval fixture is intentionally too small to motivate materialized infrastructure by runtime alone. Its purpose is to teach a correct baseline and illustrate reproducible control variables.

Project reference: P013: pandas on Snowflake teaching reference. *Not independently executed; rights review pending.*

Check. How would you distinguish an improved cached-result hit rate from a more selective scan?

12.6 Query acceleration and search optimization concepts

12.6.1 Query acceleration options

The Query Acceleration Service can provide additional compute for eligible portions of queries under supported configurations. It is not a generic speed switch for every SQL statement: eligibility, estimated benefit and consumption must be investigated. If it is activated, record service-specific credits separately from virtual-warehouse spend. A query with no qualifying work may see little improvement. The account's current feature availability, privileges and pricing must be checked before making claims in a production design.

12.6.2 Search optimization and suitable predicates

Search Optimization Service targets particular selective access patterns, including applicable point lookup and other documented predicates on large datasets. It maintains a search access path, which introduces maintenance and storage-related costs. It is inappropriate to enable solely because a report feels slow; first verify a suitable predicate, cardinality, data size and measured profile. For the small energy teaching fixture, ordinary range filtering is the correct starting point.

12.6.3 Edition, cost, and support checks

Feature support can depend on edition, object type, SQL shape, region and product evolution. A design document must state prerequisites, estimated recurring cost, pilot success metric, rollback step and service ownership. When availability is unknown, label the option a *candidate for evaluation*, not an active control.

Mechanism	Workload clue	Trade-off
Query acceleration	Eligible costly query portions	Additional service credits
Search optimisation	Selective point-like access	Access-path maintenance
Clustering	Repeated range filtering	Background reclustering cost
Larger warehouse	Memory-bound heavy compute	Higher running rate

Approval gate.
Never enable a paid optimisation feature in the companion lab without explicit approval, a bounded pilot window and a method for verifying *both* query results and metered credits. The printed code only describes read-only diagnostic queries.

Project reference: P138: alternate query-performance diagnostic reference. *Not independently executed; rights review pending.*

Check. Why should an optimisation feature's maintenance or service credits enter the final benchmark even if the query latency improves?

12.7 Improving joins, filters, and aggregations

12.7.1 Join cardinality and filter pushdown

The fastest fix is sometimes to remove work that never should have existed. Confirm the relationship and grain of every join. A many-to-many lookup can multiply energy rows and misstate totals; an accidental Cartesian join can create very large intermediates and spill. Push selective constraints toward inputs where doing so preserves semantics, and inspect operator cardinalities in Query Profile rather than assuming the optimiser behaves like handwritten execution order.

12.7.2 Aggregation and column selection

Return only columns a consumer needs; avoid `SELECT *` on wide facts when a report uses four measures. Aggregate at the approved grain rather than repeating a costly row-level join for every visual. However, an early aggregation is not safe if it discards keys needed for correct joins, changes distinctness or merges time zones incorrectly. *Performance and semantics are coupled.*

12.7.3 Rewriting without changing semantics

Listing 8: A safe baseline aggregation for the sample fixture.

```
SELECT technology, SUM(energy_mwh) AS mwh
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE interval_utc >= '2026-10-01'::TIMESTAMP_NTZ
  AND interval_utc <  '2026-10-08'::TIMESTAMP_NTZ
GROUP BY technology ORDER BY technology;
```

For the seven-day fixture, expect each technology's single-day energy multiplied by seven: Bioenergy 56, Hydro 126, Nuclear 112, Other 28, Solar 42, Wind 98, total **462.000 MWh**. The example is deliberately arithmetic, not a query-speed benchmark. Test key uniqueness after any additional dimension joins.

Rewrite review: change one idea.
Do not simultaneously change warehouse size, clustering, SQL predicates and table layout. First establish exact output equivalence; then compare the targeted physical operator. If the profile does not move, reject the hypothesis and select a new one.

Project reference: P139: Snowflake CTE Analyzer code utility. *Not independently executed; rights review pending.*

Check. Why can removing a duplicate join improve both latency and data quality?

12.8 Benchmarking performance, credits, and latency

12.8.1 Workload definition and dataset snapshots

A reproducible benchmark is a *workload contract*. Specify the dataset build and immutable snapshot, schemas and row counts, query families, parameters, selectivity, arrival distributions and concurrency. Pin the warehouse and session settings, compiler-relevant SQL, benchmark window, permissions and product version where visible. A script that runs a single small SELECT five times is a useful teaching exercise but not a production workload assessment.

Our companion includes three query families: **daily interval selection**, **per-technology total** and **fact-to-dimension lookup**. The fixture starts with the six accepted observations from prior chapters and expands them deterministically to seven dates: **42 rows / 462.000 MWh**. The input data are synthetic and extremely small; Snowflake performance on them will be dominated by overhead, caching and environment effects. Any benchmark report must say so near its headline.

Recorded field	Purpose	Sample value
Input release	Same rows in every variant	synthetic-v1
SQL version	Comparable query semantics	baseline / candidate
Warehouse	Controlled compute	approved test WH
Cache flag	Explain reuse	persisted result off
Concurrency	Reproducible load	single session
Source evidence	Trace actual run	query ID and timestamp

A production study must include representative volumes and customer-specific privacy constraints. Generating millions of synthetic rows may itself consume substantial credits; plan dataset build and cleanup separately from measurements.

Project reference: P140: cross-platform benchmark method reference. *Not independently executed; rights review pending.*

Check. Which omitted benchmark variable would make a warehouse-size comparison non-repeatable?

12.8.2 Median, p95, credits, and repeat counts

Mean runtime can hide tail latency. With a sample of at least several runs per variant, report the *median*, *p95*, minimum/maximum and sample count. State the percentile method because small samples can give unstable p95 estimates. Pair wall-clock latency with query execution and wait times; a slow BI page may not correspond to a slow query. Collect enough runs to reveal noise, alternate treatments and report uncertainty rather than choosing the best run.

A speed ratio is $S = \text{median}(t_{\text{baseline}})/\text{median}(t_{\text{candidate}})$, provided both are evaluated on equivalent inputs and correctness is established. A cost ratio is computed over identical workloads and time windows, *including* idle/overhead and any enabled paid services. An apparent improvement in one query with a maintenance-heavy clustering feature may increase total account spend.

Listing 9: Read-only history of approved tagged runs.

```
SELECT query_id, query_tag,
       total_elapsed_time / 1000.0 AS elapsed_s,
       execution_time / 1000.0 AS exec_s,
       queued_overload_time / 1000.0 AS queue_s
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_tag='book_ch12_energy_benchmark_v1'
ORDER BY start_time;
```

Illustrative numbers, not a Snowflake run.
A fabricated six-run series might yield a baseline median of roughly 1.9 s and a candidate near 1.2 s, but those values establish *nothing* about this platform. The companion flags every such CSV row as `synthetic_illustration`. Genuine results require captured query IDs, Account Usage/Snowsight evidence and billing observations.

Per-query attribution and warehouse metering answer different questions. The former may exclude idle warehouse spend; the latter must be distributed carefully under concurrent use. Label each metric's source, units, reporting lag and excluded cost classes.

Project reference: P117: Snowflake Labs data-eng-bench. *Not independently executed; rights review pending.*

Check. How can a faster median coexist with a worse p95, and which number matters for an on-call dashboard SLA?

12.8.3 Limitations and benchmark report

A defensible report separates *measurement*, *interpretation* and *decision*. A measurement states the environment, version, observed numbers and query IDs. An interpretation states the most likely mechanism and alternative explanations. A decision says whether to deploy, revert or collect more data, and names the owner and rollback path.

In the accompanying report template, explicitly distinguish the reproducible **offline correctness oracle** from the **fabricated performance demonstration CSV** and **real Snowflake metrics still to be collected**. No numerical speedup, p95 or credit saving for Snowflake is claimed by this chapter. If the pilot is ever run, add actual query IDs and a timestamp before replacing the placeholders.

Gate	Required evidence	Fail response
Correctness	Same keyed result set	Stop experiment
Repeatability	Run count; stable snapshot	Collect more runs
Diagnosis	Scan/queue/spill change	Reject hypothesis
Economics	Credits and auxiliary charges	Estimate TCO again
Deployability	Owner, permission, rollback	Do not enable feature

Common bias. Selecting only queries that benefit a proposed warehouse upgrade creates an invalid cost argument. Include ordinary report queries, expensive peak-time queries and repeated user interactions. If the dataset is small, state that warehouse provisioning and fixed overhead dominate; extrapolation to hundreds of millions of records would be misleading.

The most useful outcome may be "do not optimise".
If p95 already meets the requirement, the warehouse is inexpensive, and the query is correct, a new maintenance-intensive feature may decrease reliability for negligible benefit. Recording a rational no-change decision is successful performance engineering.

Project reference: P141: Snowflake FinOps optimisation example. *Not independently executed; rights review pending.*

Check. What two facts must be established before claiming a percent reduction in Snowflake credit spending?

12.L Guided lab, chapter review, and exercises

12.L.1 Prerequisites, inputs, environment, permissions, and cost guardrails

Lab mission: compare two semantically equivalent energy-report SQL patterns, produce a correctness certificate and prepare a measurement plan. The companion includes `benchmark_queries.sql`, `energy_sample.csv`, `benchmark_results.csv`, `offline_validate.py` and `tuning_report.md`. The results CSV is **synthetic illustration only**: it was not extracted from any warehouse.

Offline route (no cost): Python 3 standard library; run the offline validator and the Python unit-test command listed in the companion README. **Cloud option (billable):** only with an approved isolated database and dedicated small test warehouse, least-privilege role, auto-suspend, verified budget and explicit cleanup plan. Never run optional DDL on a shared/production account.

12.L.2 Step-by-step procedure and selected code

Step 1. Validate six source energy values, their uniqueness and **66.000 MWh** single-day sum. **Step 2.** Expand the same six values across seven UTC dates for a **42-row / 462.000 MWh** reproducible exercise. **Step 3.** Examine two query shapes: a function-based date predicate and a half-open range; confirm their resulting aggregates agree for every requested day. **Step 4.** If authorised, capture multiple tagged Query History IDs and inspect Query Profile. **Step 5.** Record medians and p95 only from actual measurements, not from the demonstration CSV.

Listing 10: Sample range query from the companion: 4 October only.

```
SELECT technology, SUM(energy_mwh) AS mwh
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE interval_utc >= '2026-10-04'::TIMESTAMP_NTZ
  AND interval_utc <  '2026-10-05'::TIMESTAMP_NTZ
GROUP BY technology ORDER BY technology;
```

Expected aggregate for 4 October: **66.000 MWh**. *Performance expectation: none.* The fixture has no meaningful partition-scale power to demonstrate a speed improvement; query timing is environmental.

Output folder
Submit an oracle test log, versioned benchmark SQL, environment/privilege declaration, result rows and Query IDs (if actually measured), a comparison table and an ADR stating why a change is accepted, rejected or postponed.

12.L.3 Expected results, assertions, failure diagnosis, and cleanup

Expected *offline* invariants: six distinct technologies per date; exactly seven dates and 42 accepted fact rows; daily generation 66.000 MWh; total 462.000 MWh; both date-filter formulations have the same keyed output; no negative energy; and the benchmark demonstration file is explicitly marked as synthetic. Failures can reveal mistaken inclusive end boundaries, duplicated joins, unstable fixture order or incorrectly treated null values. Repair the correctness model before comparing speed.

Cleanup. Reset QUERY_TAG and the result-cache session override. With authorised ownership only, remove lab-specific tables and suspend the isolated warehouse after capturing evidence. The companion does not automatically drop objects. Do not remove data or grants belonging to other work.

12.L.4 Foundation exercise, practitioner exercise, and production-design decision

1. **Foundation (20 points).** Explain persisted result cache versus warehouse-local cache and give two sources of apparent timing improvement that are *not* SQL optimisation.
2. **Practitioner (30 points).** Design a seven-day query-equivalence test, collect sufficient tagged runs on an authorised Snowflake lab (or write *not measured*), and record median, p95, partitions scanned and wait/spill indicators.
3. **Architectural decision (30 points).** A Power BI dashboard slows at peak usage. Compare query rewrite, warehouse resizing, multi-cluster, clustering and a serving mart; specify the evidence necessary to reject each option.
4. **Evidence and safety (20 points).** State account permissions, budget guardrails, data provenance, rollback, time window, reviewer and unresolved limitations.

Chapter review. (1) Why must query correctness precede tuning? (2) What distinguishes provisioning from overload wait? (3) When can clustering be cost-negative? (4) Why is one benchmark run insufficient? (5) Which credits are missing from per-query execution attribution?

Chapter synthesis

A credible tuning claim has three legs: *an equal correct answer, a repeatable explanation of the bottleneck, and a measured overall cost.* Remove one leg and the conclusion becomes guesswork. Chapter 13 extends this discipline to governance, reliability, access and cost ownership.

Authoritative technical references (reviewed 22 Oct 2026). Snowflake Documentation: Query History; Query Profile / Snowsight Activity; Clustering Keys; Query Acceleration Service; Search Optimization Service; Warehouse considerations; Account Usage Query Attribution. Query History; Query Profile; Clustering; Acceleration. GitHub repositories cited above are external teaching references, *not* executed or licensed for republication in this chapter.